

ai面板识别

题目描述

AI识别到面板上有 N ($1 \leq N \leq 100$) 个指示灯，灯大小一样，任意两个之间无重叠。

由于 AI 识别误差，每次别到的指示灯位置可能有差异，以4个 **坐标值** 描述AI识别的指示灯的大小和位置(左上角 x_1, y_1 ，右下角 x_2, y_2)，

请输出先行后列排序的指示灯的编号，排序规则：

1. 每次在尚未排序的灯中挑选最高的灯作为的基准灯，
2. 找出和基准灯属于同一行所有的灯进行排序。两个灯高低偏差不超过灯半径算同一行（即两个灯坐标的差 $\leq$ 灯高度的一半）。

输入描述

第一行为 N ，表示灯的个数

接下来 N 行，每行为1个灯的坐标信息，格式为：

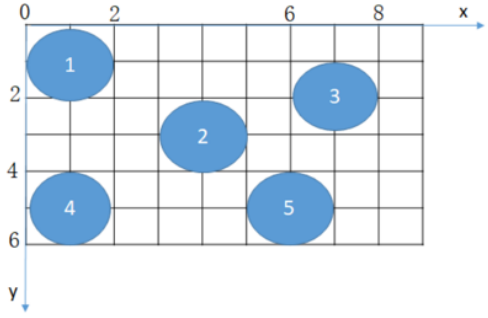
编号 x_1 y_1 x_2 y_2

- 编号全局唯一
- $1 \leq \text{编号} \leq 100$
- $0 \leq x_1 < x_2 \leq 1000$
- $0 \leq y_1 < y_2 \leq 1000$

输出描述

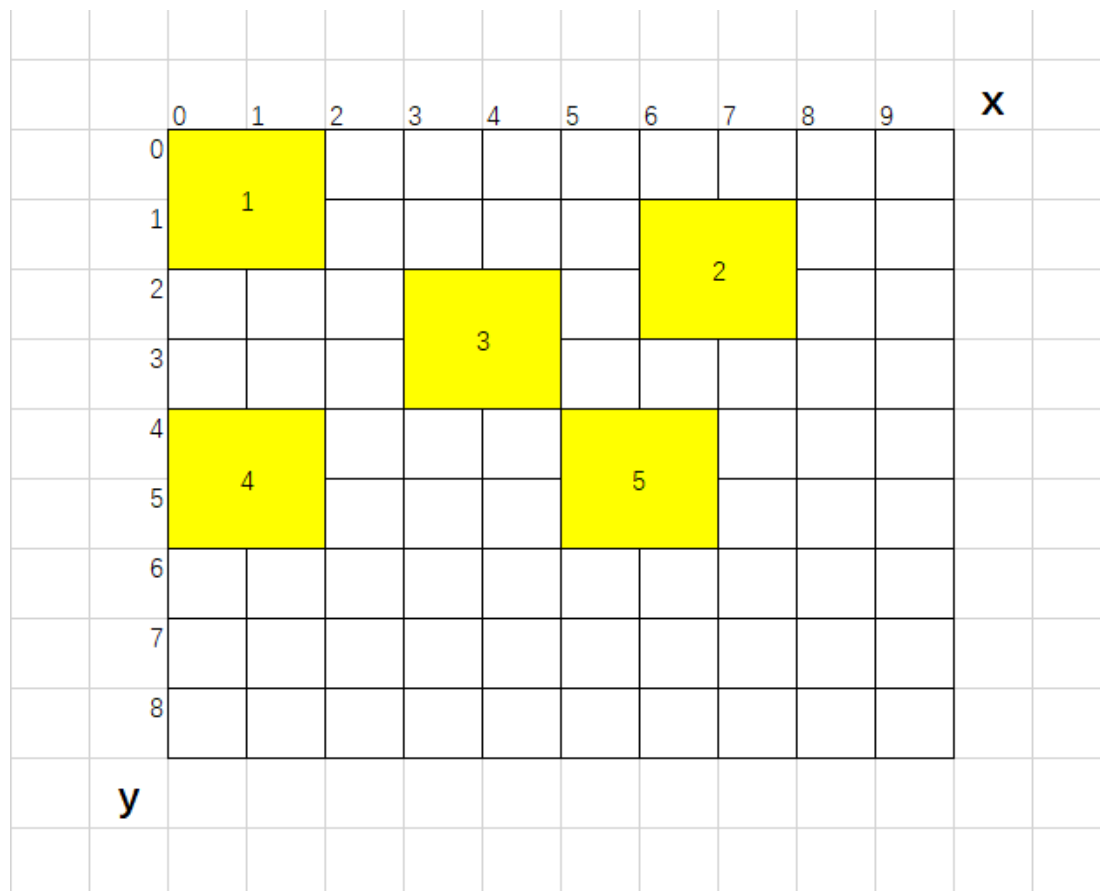
排序后的编号列表，编号之间以空格分隔

用例

输入	5 1 0 0 2 2 2 6 1 8 3 3 3 2 5 4 5 5 4 7 6 4 0 4 2 6
输出	1 2 3 4 5
说明	

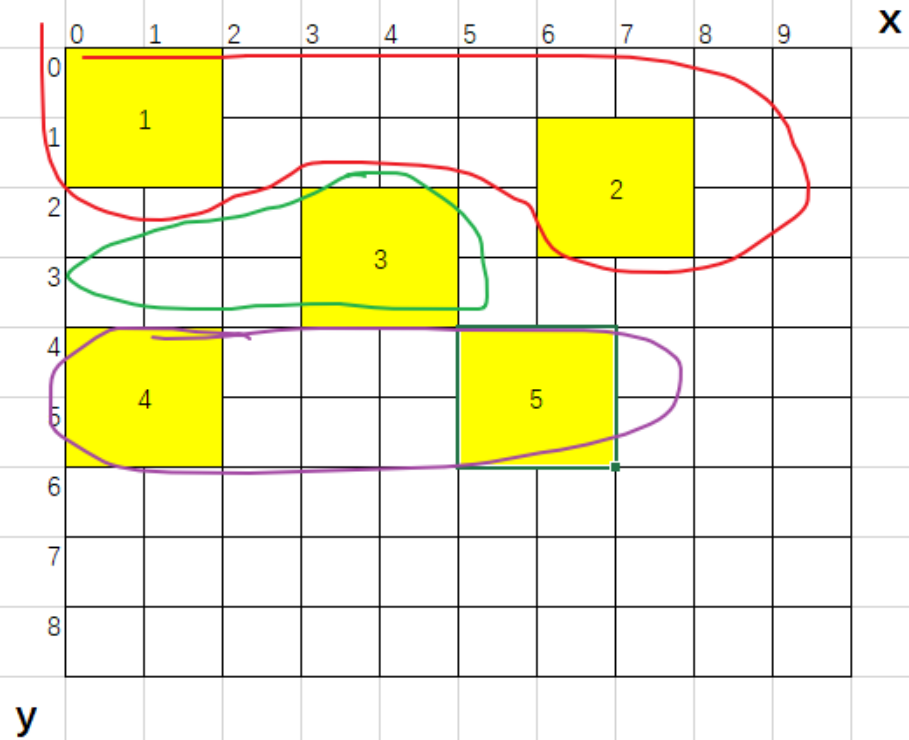
题目解析

题目给的图好像有问题，我画出来是



按照题目意思，每次取出一个最高的灯（y坐标最小的），然后找出和最高灯坐标相差小于等于灯半径的，作为同一行，然后按照x轴坐标进行升序

即存在如下用圈框住的三行，因此打印顺序是：1，2，3，4，5



Java算法源码

```
1 import java.util.ArrayList;
2 import java.util.Arrays;
3 import java.util.Scanner;
4 import java.util.StringJoiner;
5
6 public class Main {
7     public static void main(String[] args) {
8         Scanner sc = new Scanner(System.in);
```

```

9
10 |     int n = sc.nextInt();
11
12 |     Light[] lights = new Light[n];
13 |     for (int i = 0; i < n; i++) {
14 |         int id = sc.nextInt();
15 |         int x1 = sc.nextInt();
16 |         int y1 = sc.nextInt();
17 |         int x2 = sc.nextInt();
18 |         int y2 = sc.nextInt();
19 |         lights[i] = new Light(id, (x1 + x2) / 2, (y1 + y2) / 2, (x2 - x1) / 2);
20 |     }
21
22 |     System.out.println(getResult(lights));
23 | }
24
25 | public static String getResult(Light[] lights) {
26 |     // 按照圆心y坐标升序
27 |     Arrays.sort(lights, (a, b) -> a.y - b.y);
28
29 |     // ans记录题解
30 |     StringJoiner ans = new StringJoiner(" ");
31
32 |     // sameRowLights记录同一行的灯
33 |     ArrayList<Light> sameRowLights = new ArrayList<>();
34 |     Light base = lights[0];
35 |     sameRowLights.add(base);
36
37 |     for (int i = 1; i < lights.length; i++) {
38 |         Light light = lights[i];
39
40 |         // 如果lights[i]的纵坐标和base的纵坐标相差不超过半径，则视为同一行
41 |         if (light.y - base.y <= base.r) {
42 |             sameRowLights.add(light);
43 |         } else {
44 |             // 否则，不是同一行
45 |             // 针对同一行的灯，再按照横坐标升序

```

```

46         sameRowLights.sort((a, b) -> a.x - b.x); 47
48         sameRowLights.clear();
49
50         // 开始新的一行记录
51         base = light;
52         sameRowLights.add(base);
53     }
54 }
55
56 // 注意不要漏了最后一行
57 if (sameRowLights.size() > 0) {
58     sameRowLights.sort((a, b) -> a.x - b.x);
59     sameRowLights.forEach(a -> ans.add(a.id + ""));
60 }
61
62 return ans.toString();
63 }
64 }
65
66 class Light {
67     int id; // 编号
68     int x; // 圆心横坐标
69     int y; // 圆心纵坐标
70     int r; // 圆半径
71
72     public Light(int id, int x, int y, int r) {
73         this.id = id;
74         this.x = x;
75         this.y = y;
76         this.r = r;
77     }
78 }

```

运行

```
1  const rl = require("readline").createInterface({ input: process.stdin });
2  var iter = rl[Symbol.asyncIterator]();
3  const readline = async () => (await iter.next()).value;
4
5  class Light {
6      constructor(id, x, y, r) {
7          this.id = id; // 编号
8          this.x = x; // 圆心横坐标
9          this.y = y; // 圆心纵坐标
10         this.r = r; // 圆半径
11     }
12 }
13
14 void (async function () {
15     const n = parseInt(await readline());
16
17     const lights = [];
18     for (let i = 0; i < n; i++) {
19         const [id, x1, y1, x2, y2] = (await readline()).split(" ").map(Number);
20         lights.push(new Light(id, (x1 + x2) >> 1, (y1 + y2) >> 1, (x2 - x1) >> 1));
21     }
22
23     console.log(getResult(lights));
24 })();
25
26 function getResult(lights) {
27     // 按照圆心y坐标升序
28     lights.sort((a, b) => a.y - b.y);
29
30     // ans记录题解
31     const ans = [];
32
33     // sameRowLights记录同一行的灯
34     const sameRowLights = [];
35     let base = lights[0];
36     sameRowLights.push(base);
```

```

37 | 38 |   for (let i = 1; i < lights.length; i++) {
39 |     const light = lights[i];
40 |
41 |     // 如果lights[i]的纵坐标和base的纵坐标相差不超过半径，则视为同一行
42 |     if (light.y - base.y <= base.r) {
43 |       sameRowLights.push(light);
44 |     } else {
45 |       // 否则，不是同一行
46 |       // 针对同一行的灯，再按照横坐标升序
47 |       sameRowLights.sort((a, b) => a.x - b.x).forEach((a) => ans.push(a.id));
48 |       sameRowLights.length = 0;
49 |
50 |       // 开始新的一行记录
51 |       base = light;
52 |       sameRowLights.push(base);
53 |     }
54 |   }
55 |
56 |   // 注意不要漏了最后一行
57 |   if (sameRowLights.length > 0) {
58 |     sameRowLights.sort((a, b) => a.x - b.x).forEach((a) => ans.push(a.id));
59 |     sameRowLights.length = 0;
60 |   }
61 |
62 |   return ans.join(" ");
63 | }

```

Python算法源码

```

1 | class Light:
2 |     def __init__(self, id, x, y, r):
3 |         self.id = id # 编号
4 |         self.x = x # 圆心横坐标

```

```

5         self.y = y # 圆心纵坐标 6         self.r = r # 圆半径
7
8
9 # 输入获取
10 n = int(input())
11 arr = [list(map(int, input().split())) for _ in range(n)]
12 lights = list(map(lambda ele: Light(ele[0], (ele[1] + ele[3]) // 2, (ele[2] + ele[4]) // 2, (ele[3] - ele[1]) // 2), arr))
13
14
15 # 算法入口
16 def getResult():
17     # 按照圆心y坐标升序
18     lights.sort(key=lambda l: l.y)
19
20     # ans记录题解
21     ans = []
22
23     # sameRowLights记录同一行的灯
24     sameRowLights = []
25     base = lights[0]
26     sameRowLights.append(base)
27
28     for i in range(1, len(lights)):
29         light = lights[i]
30
31         # 如果lights[i]的纵坐标和base的纵坐标相差不超过半径，则视为同一行
32         if light.y - base.y <= base.r:
33             sameRowLights.append(light)
34         else:
35             # 否则，不是同一行
36             # 针对同一行的灯，再按照横坐标升序
37             sameRowLights.sort(key=lambda l: l.x)
38             for l in sameRowLights:
39                 ans.append(l.id)
40             sameRowLights.clear()
41
42     # 开始新的一行记录

```

```

43         base = light
44         sameRowLights.append(base)
45
46     # 注意不要漏了最后一行
47     if len(sameRowLights) > 0:
48         sameRowLights.sort(key=lambda l: l.x)
49         for l in sameRowLights:
50             ans.append(l.id)
51
52     return " ".join(map(str, ans))
53
54
55 # 算法调用
56 print(getResult())

```

C算法源码

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  #define MAX_SIZE 100
6
7  // 灯信息结构体
8  typedef struct {
9      int id; // 灯id
10     int x; // 灯的横坐标
11     int y; // 灯的纵坐标
12     int r; // 灯的半径
13 } Light;
14
15 // 记录题解
16 char res[100000];
17

```

运行

```

18 | int cmp(const void *a, const void *b) {
19 |     // 按照灯的圆心y坐标升序
20 |     return ((Light *) a)->y - ((Light *) b)->y;
21 | }
22 |
23 | int cmp2(const void *a, const void *b) {
24 |     // 针对同一行的灯，按照圆心x坐标升序
25 |     return ((Light *) a)->x - ((Light *) b)->x;
26 | }
27 |
28 | void joinToRes(Light sameRowLights[], int sameRowLights_size) {
29 |     qsort(sameRowLights, sameRowLights_size, sizeof(Light), cmp2);
30 |
31 |     for (int j = 0; j < sameRowLights_size; j++) {
32 |         char tmp[100];
33 |         sprintf(tmp, "%d", sameRowLights[j].id);
34 |         strcat(res, tmp);
35 |         strcat(res, " ");
36 |     }
37 | }
38 |
39 | int main() {
40 |     // 灯的个数
41 |     int n;
42 |     scanf("%d", &n);
43 |
44 |     Light lights[MAX_SIZE];
45 |     for (int i = 0; i < n; i++) {
46 |         // 灯编号，以及坐标信息
47 |         int id, x1, y1, x2, y2;
48 |         scanf("%d %d %d %d %d", &id, &x1, &y1, &x2, &y2);
49 |
50 |         lights[i].id = id;
51 |         lights[i].x = (x1 + x2) / 2;
52 |         lights[i].y = (y1 + y2) / 2;
53 |         lights[i].r = (x2 - x1) / 2;
54 |     }

```

```
55 |
56 |     // 同一行的灯集合
57 |     Light sameRowLights[MAX_SIZE];
58 |     int sameRowLights_size = 0;
59 |
60 |     // 按照圆心y坐标升序
61 |     qsort(lights, n, sizeof(Light), cmp);
62 |
63 |     // sameRowLights记录同一行的灯
64 |     Light base = lights[0];
65 |     sameRowLights[sameRowLights_size++] = base;
66 |
67 |     for (int i = 1; i < n; i++) {
68 |         Light light = lights[i];
69 |
70 |         // 如果lights[i]的纵坐标和base的纵坐标相差不超过半径，则视为同一行
71 |         if (light.y - base.y <= base.r) {
72 |             sameRowLights[sameRowLights_size++] = light;
73 |         } else {
74 |             // 否则，不是同一行
75 |             // 针对同一行的灯，再按照横坐标升序
76 |             joinToRes(sameRowLights, sameRowLights_size);
77 |
78 |             // 开始新的一行记录
79 |             sameRowLights_size = 0;
80 |             base = light;
81 |             sameRowLights[sameRowLights_size++] = base;
82 |         }
83 |     }
84 |
85 |     // 注意不要漏了最后一行
86 |     if (sameRowLights_size > 0) {
87 |         joinToRes(sameRowLights, sameRowLights_size);
88 |     }
89 |
90 |     // joinToRes中会在res最后多拼接一个空格，这里去掉
91 |     res[strlen(res) - 1] = '\0';
92 |
```

```
93     puts(res);
94
95     return 0;
96 }
```

,

C++算法源码

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  class Light {
6  public:
7      int id; // 编号
8      int x; // 圆心横坐标
9      int y; // 圆心纵坐标
10     int r; // 圆半径
11
12     Light(int id, int x, int y, int r) : id(id), x(x), y(y), r(r) {};
13 };
14
15 int main() {
16     int n;
17     cin >> n;
18
19     vector<Light> lights;
20     for (int i = 0; i < n; i++) {
21         int id, x1, y1, x2, y2;
22         cin >> id >> x1 >> y1 >> x2 >> y2;
23
24         lights.emplace_back(id, (x1 + x2) / 2, (y1 + y2) / 2, (x2 - x1) / 2);
25     }
26
27     // 按照圆心y坐标升序
```

```

28 | sort(lights.begin(), lights.end(), [](Light a, Light b) {
29 |     return a.y < b.y;30 | });
31 |
32 | Light base = lights[0];
33 |
34 | // sameRowLights记录同一行的灯
35 | vector<Light> sameRowLights{base};
36 |
37 | for (int i = 1; i < lights.size(); i++) {
38 |     Light light = lights[i];
39 |
40 |     if (light.y - base.y <= base.r) {
41 |         // 如果lights[i]的纵坐标和base的纵坐标相差不超过半径，则视为同一行
42 |         sameRowLights.emplace_back(light);
43 |     } else {
44 |         // 否则，不是同一行
45 |         // 针对同一行的灯，再按照横坐标升序
46 |         sort(sameRowLights.begin(), sameRowLights.end(), [](Light a, Light b) {
47 |             return a.x < b.x;
48 |         });
49 |
50 |         for (const auto &item: sameRowLights) {
51 |             cout << item.id << " ";
52 |         }
53 |
54 |         sameRowLights.clear();
55 |
56 |         // 开始新的一行记录
57 |         base = light;
58 |         sameRowLights.emplace_back(base);
59 |     }
60 | }
61 |
62 | // 注意不要漏了最后一行
63 | if (!sameRowLights.empty()) {
64 |     sort(sameRowLights.begin(), sameRowLights.end(), [](Light a, Light b) {

```

```
65 |         return a.x < b.x;66 |     });
67 |
68 |     for (const auto &item: sameRowLights) {
69 |         cout << item.id << " ";
70 |     }
71 | }
72 |
73 | return 0;
74 | }
```